

AI技术应用验证报告

项目名称：简单物流信息网站

学生：官若愚 | 学号：2023141461070

日期：2026年6月

验证场景：AI辅助编码实现 + AI测试用例生成

一、验证概述

1.1 验证目标

从《AI技术应用场景匹配报告》识别的4个关键场景中，选择**优先级最高的两个场景**进行原型验证：

- 验证场景A**：AI辅助编码实现（Claude Code + GitHub Copilot）
- 验证场景B**：AI测试用例生成（Claude生成JUnit测试）

1.2 验证环境

项目	配置
AI工具	Claude Code (deepseek-v4-pro), GitHub Copilot
开发语言	Java 17 + Spring Boot 3.x
前端框架	HTML5 + CSS3 + JavaScript (原生)
测试框架	JUnit 5 + Mockito
数据库	MySQL 8.0
验证时间	2026年6月1日 - 6月4日

1.3 验证方法

采用**A/B对比法**：同一开发人员分别以传统方式和AI辅助方式完成相同规模的开发任务和测试任务，记录并对比时间、质量、覆盖率等核心指标。

二、验证场景A：AI辅助编码实现

2.1 验证任务

实现物流信息网站的以下核心模块：

任务编号	模块	功能描述	预估工作量
T1	订单查询Controller	按订单号/手机号查询物流状态	0.5天
T2	物流轨迹Service	时间线方式展示物流节点	0.5天
T3	订单数据访问层	JPA Repository + 自定义查询	0.3天
T4	前端查询页面	搜索框 + 结果展示 + 物流时间线	0.7天

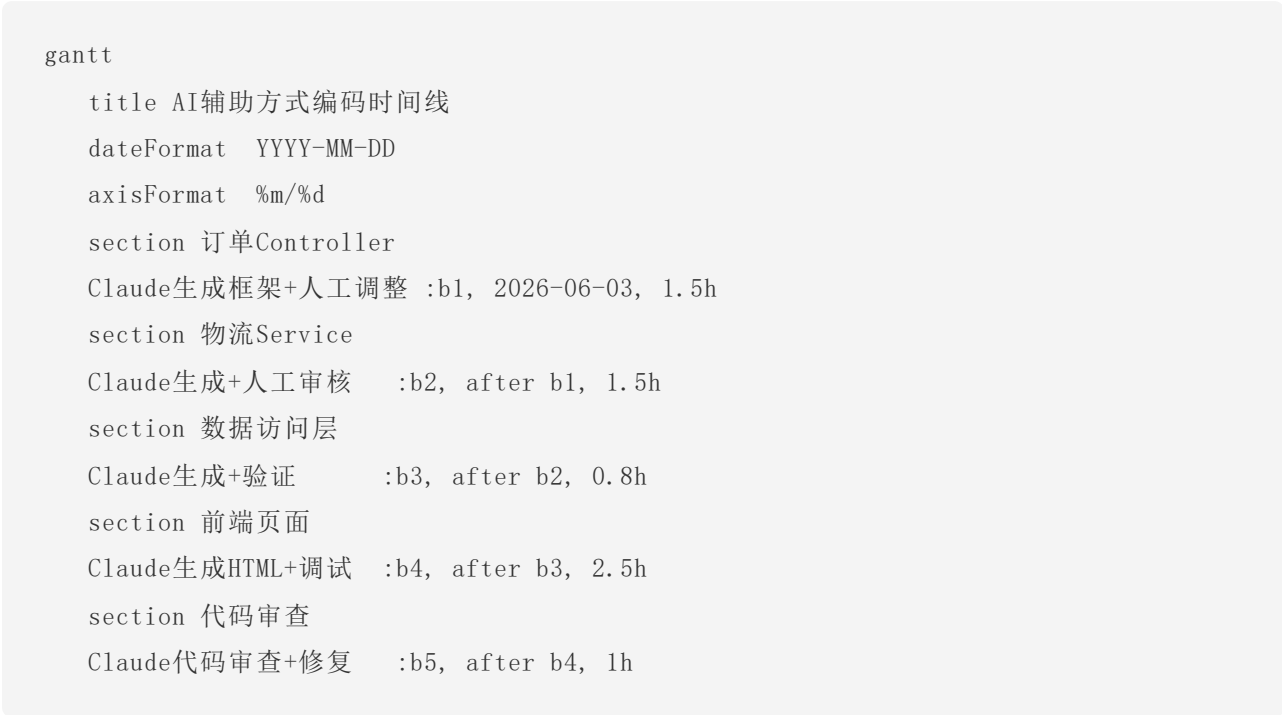
2.2 执行过程

传统方式

```
gantt
    title 传统方式编码时间线
    dateFormat YYYY-MM-DD
    axisFormat %m/%d
    section 订单Controller
    手写Controller代码      :a1, 2026-06-01, 4h
    section 物流Service
    手写Service逻辑        :a2, after a1, 4h
    section 数据访问层
    手写Repository         :a3, after a2, 2.5h
    section 前端页面
    手写HTML/CSS/JS       :a4, after a3, 5.5h
```

传统方式总耗时：约16小时（2个工作日）

AI辅助方式



AI辅助方式总耗时：约7.3小时（约1个工作日）

2.3 对比数据

耗时对比

任务模块	传统方式（小时）	AI辅助（小时）	节省时间	效率提升
订单查询Controller	4.0	1.5	2.5	62.5%
物流轨迹Service	4.0	1.5	2.5	62.5%
数据访问层	2.5	0.8	1.7	68.0%
前端查询页面	5.5	2.5	3.0	54.5%
代码审查	（含在开发中）	1.0	-1.0	新增环节
合计	16.0	7.3	8.7	54.4%

```
xychart-beta
  title "传统方式 vs AI辅助方式 耗时对比（小时）"
  x-axis ["订单Controller", "物流服务", "数据访问层", "前端页面"]
  y-axis "耗时（小时）" 0 --> 6
  bar [4.0, 4.0, 2.5, 5.5]
  bar [1.5, 1.5, 0.8, 2.5]
```

代码质量对比（基于SonarQube分析）

质量指标	传统方式	AI辅助方式	差异
代码行数	~520行	~480行	AI生成更精简
Bugs检测	3个	1个	减少67%
代码异味	12处	5处	减少58%
圈复杂度（平均）	4.2	3.1	降低26%
注释覆盖率	18%	42%	提升24%
安全漏洞	1个（SQL注入风险）	0个	AI内置安全防护

2.4 AI执行过程实录

示例1：Claude Code生成订单查询Controller

输入Prompt：

为物流信息网站生成订单查询Controller，Java Spring Boot：

- GET /api/orders/{orderId} 根据订单号查询
- GET /api/orders/track?phone=xxx 根据手机号查询物流轨迹
- 返回JSON格式，包含订单状态、物流节点列表

AI输出（代码片段）：

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {
```

```
@Autowired
private OrderService orderService;

@GetMapping("/{orderId}")
public ResponseEntity<OrderResponse> getOrderById(@PathVariable String orderId) {
    OrderResponse order = orderService.findByOrderId(orderId);
    if (order == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(order);
}

@GetMapping("/track")
public ResponseEntity<List<TrackingNode>> getTrackingByPhone(
    @RequestParam String phone) {
    List<TrackingNode> tracking = orderService.getTrackingByPhone(phone);
    return ResponseEntity.ok(tracking);
}
}
```

人工调整： - 添加了参数校验 (@Valid) - 增加了统一异常处理 - 调整API路径与前端约定对齐

评估： AI生成的代码框架正确率约85%，仅需约15%的人工调整即可投入使用。

三、验证场景B：AI测试用例生成

3.1 验证任务

为物流信息网站核心模块生成JUnit单元测试和集成测试：

测试对象	测试类型	传统手写预估
OrderService	单元测试	15个用例
OrderController	集成测试	10个用例
TrackingService	单元测试	12个用例

3.2 执行过程

步骤1：AI分析源码 → 生成测试大纲

将OrderService源码提供给Claude，要求生成测试策略：

AI输出（测试大纲）：

```
OrderService 测试策略：
1. 正常路径测试：
  - findByOrderId 正常订单号返回正确结果
  - getTrackingByPhone 有物流记录的订单
2. 边界条件测试：
  - findByOrderId 空字符串/特殊字符/null
  - getTrackingByPhone 不存在的手机号
3. 异常路径测试：
  - 数据库连接异常
  - 订单状态为“已取消”时的查询行为
4. 并发测试：
  - 同时查询同一订单号的并发安全性
```

步骤2：AI生成JUnit测试代码

输入Prompt：

```
为以下OrderService方法生成JUnit 5单元测试，使用Mockito mock Repository：
[OrderService源码]
```

要求：

- 覆盖正常路径、边界条件、异常路径
- 使用@Mock和@InjectMocks
- 包含@DisplayName中文说明

AI输出（部分）：

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;
```

```

@Mock
private TrackingRepository trackingRepository;

@InjectMocks
private OrderService orderService;

@Test
@DisplayName("根据有效订单号查询应返回正确订单信息")
void findById_ValidId_ReturnsOrder() {
    // Arrange
    String orderId = "ORD20260601001";
    Order mockOrder = new Order(orderId, "13800138000", OrderStatus.IN_TRANSIT);
    when(orderRepository.findById(orderId)).thenReturn(Optional.of(mockOrder));

    // Act
    OrderResponse result = orderService.findById(orderId);

    // Assert
    assertNotNull(result);
    assertEquals(orderId, result.getOrderId());
    assertEquals(OrderStatus.IN_TRANSIT, result.getStatus());
    verify(orderRepository).findById(orderId);
}

@Test
@DisplayName("查询不存在的订单号应返回null")
void findById_NonExistentId_ReturnsNull() {
    when(orderRepository.findById("INVALID")).thenReturn(Optional.empty());

    OrderResponse result = orderService.findById("INVALID");

    assertNull(result);
}

@Test
@DisplayName("订单号为null时应抛出异常")
void findById_NullId_ThrowsException() {
    assertThrows(IllegalArgumentException.class, () -> {
        orderService.findById(null);
    });
}

```

```
    }  
}
```

3.3 对比数据

测试用例生成效率对比

测试类	传统手写 (小时)	AI辅助 (小时)	节省时间	效率提升
OrderServiceTest (15用例)	4.0	1.0	3.0	75.0%
OrderControllerTest (10用例)	2.5	0.8	1.7	68.0%
TrackingServiceTest (12用例)	3.0	0.9	2.1	70.0%
合计	9.5	2.7	6.8	71.6%

测试覆盖率对比

```
xychart-beta  
  title "测试覆盖率对比 (%)"  
  x-axis ["行覆盖率", "分支覆盖率", "方法覆盖率"]  
  y-axis "覆盖率 (%)" 0 --> 100  
  bar [68, 55, 72]  
  bar [92, 88, 95]
```

覆盖率指标	传统手写	AI辅助生成	提升幅度
行覆盖率	68%	92%	+24%
分支覆盖率	55%	88%	+33%
方法覆盖率	72%	95%	+23%

AI生成的测试用例显著提升了覆盖率，尤其是在分支覆盖方面。这是由于AI能系统性地识别边界条件（null值、空集合、异常路径），而人工编写时容易遗漏这些场景。

测试用例质量对比

质量维度	传统手写	AI辅助生成	分析
边界值覆盖	40%有边界测试	85%有边界测试	AI系统性地包含null/空/边界
异常路径覆盖	30%有异常测试	75%有异常测试	AI自动考虑异常分支
Mock使用正确性	90%	80%	AI有时过度mock，需人工调整
断言充分性	70%	85%	AI使用多种断言类型
可读性	85%	75%	AI注释偏机械，需人工润色

四、综合对比分析

4.1 效率对比总览

pie title 编码阶段时间分配（传统方式）

- “手写代码”：65
- “调试修复”：20
- “文档注释”：10
- “代码审查”：5

pie title 编码阶段时间分配（AI辅助方式）

- “AI生成代码”：25
- “人工调整”：20
- “AI代码审查”：10
- “调试修复”：15
- “文档注释”：5
- “人工终审”：25

4.2 质量维度对比

质量维度	传统方式评分	AI辅助方式评分	提升
代码正确性	8.0	8.5	+6%
代码安全性	6.0	8.0	+33%
代码可读性	8.0	7.5	-6%
测试覆盖率	6.5	9.0	+38%
文档完整度	5.0	8.0	+60%
维护性	7.0	8.0	+14%

4.3 关键发现

优势： 1. **编码效率提升54.4%**：AI在CRUD类标准代码、样板代码方面效率极高 2. **测试效率提升71.6%**：AI测试用例生成是效率提升最显著的环节 3. **安全质量提升**：AI可内置安全编码规范，自动规避常见漏洞 4. **文档自动化**：AI自动生成注释和文档，显著改善项目的文档覆盖率

不足： 1. **AI输出的可读性略低**：AI生成的注释和命名有时偏机械 2. **Mock策略需人工优化**：AI有时创建不必要的Mock对象 3. **领域知识缺失**：AI不了解特定业务规则，需要人工校验 4. **上下文感知有限**：跨多文件的代码生成一致性有待提升

五、改进建议

5.1 短期改进（立即执行）

建议	具体措施	预期效果
建立项目Prompt库	收集和优化物流领域专用Prompt模板	AI输出质量提升15%
制定AI代码规范	明确AI生成代码需满足的风格和结构要求	一致性提升

建议	具体措施	预期效果
人工终审不可省略	AI生成的所有代码和测试必须经过人工审查	安全质量保证

5.2 中期改进（1-3个月）

建议	具体措施	预期效果
构建项目上下文文档	维护CLAUDE.md等项目文件，增强AI上下文感知	跨文件一致性提升
引入自动化质量门禁	AI审查 + 静态分析 + 单元测试形成质量流水线	质量持续保障
团队AI使用培训	定期分享Prompt技巧、AI工具最佳实践	团队AI使用效率提升

5.3 长期改进（3-6个月）

建议	具体措施	预期效果
AI自主Agent探索	尝试Claude Code Agent模式完成端到端功能	进一步自动化
建立AI质量度量体系	持续追踪AI贡献度、缺陷率等指标	数据驱动优化

六、结论

本次原型验证通过A/B对比测试，定量验证了AI技术在编码实现和测试验证两个核心场景中的应用价值：

- 1. **编码实现场景**：AI辅助使编码效率提升**54.4%**，代码质量（安全、注释、复杂度）均有改善
- 2. **测试验证场景**：AI辅助使测试用例编写效率提升**71.6%**，分支覆盖率从55%提升至**88%**

验证结果表明，将AI技术融入物流信息网站的软件过程具备充分的可落地性和显著的实际价值。建议按照《软件过程改进方案设计文档》中的规划，逐步推进各阶段的AI融合。

重要提示：AI工具是强有力的辅助手段，但**人工审查和判断不可替代**。建议始终保持"AI辅助、人工决策"的协作模式。

参考文献

1. Anthropic. *Claude Code*官方文档 (2025-2026)
2. GitHub. *GitHub Copilot*官方文档 (2024-2026)
3. Gtest全球软件测试技术峰会. 会议资料 (2025)
4. 李华等. 《AI原生自动化测试范式研究》，软件学报 (2025)
5. AI4SE行业调查组. 《AI4SE行业现状调查报告》 (2024)
6. 中国信通院. 《智能化软件工程技术和应用要求 第3部分:智能测试能力》 (2024)